

Домашняя работа 03

1. Создайте новую базу данных и перейдите в неё.
2. Разработайте таблицу для бизнес-кейса "Меню ресторана" с минимум пятью полями. Наполните таблицу данными, используя модификаторы (например, Nullable, LowCardinality), где это необходимо. Не забудьте добавить комментарии к полям.
3. Протестируйте выполнение операций CRUD на созданной таблице.
4. Добавьте несколько новых полей в таблицу и удалите два-три существующих.
5. Выполните выборку данных (select) из любой таблицы из [sample dataset](#)
6. Материализуйте выбранную таблицу, создав её копию в виде отдельной таблицы.
7. Попрактикуйтесь с партициями: выполните операции ATTACH, DETACH и DROP. После этого добавьте новые данные в первоначально созданную таблицу.

1.Создайте новую базу данных и перейдите в неё.

```
create database if not exists hw03 comment 'Домашняя работа №3';
```

2.Разработайте таблицу для бизнес-кейса "Меню ресторана" с минимум пятью полями. Наполните таблицу данными, используя модификаторы (например, Nullable, LowCardinality), где это необходимо. Не забудьте добавить комментарии к полям.

```
create database if not exists hw03 comment 'Домашняя работа №3';
```

```
create table IF not EXISTS hw03.menu
```

```
(
  id UInt32 comment 'Идентификатор строки',
  menu_category LowCardinality(String) comment 'Категория блюда',
  menu_type LowCardinality(String) comment 'Тип блюда',
  name String comment 'Название блюда',
  country LowCardinality(String) default 'не задано' comment 'Национальное блюдо',
  price UInt32 default 0
)
```

```
ENGINE = ReplacingMergeTree
```

```
PARTITION BY menu_category
```

```
ORDER BY (menu_type,
  country,
  name)
```

```
SETTINGS index_granularity = 8192;
```

```
comment 'Таблица меню ресторана';
```

```
INSERT INTO hw03.menu (id, menu_category, menu_type, name, country, price) VALUES
```

```
(1, 'Салаты', 'Вегетарианское', 'Салат Цезарь', 'Италия', 500),
(2, 'Десерты', 'Торт', 'Шоколадный торт', 'Франция', 300),
(3, 'Супы', 'Холодное', 'Окрошка на кефире', 'Россия', 150),
(4, 'Супы', 'Холодное', 'Окрошка на квасе', 'Россия', 150),
(5, 'Основное блюдо', 'Мясные блюда', 'Утка по-пекински', 'Китай', 5000),
(6, 'Основное блюдо', 'Мясные блюда', 'Пельмени', 'Россия', 300);
```

3.Протестируйте выполнение операций CRUD на созданной таблице.

```
INSERT INTO hw03.menu (id, menu_category, menu_type, name, country, price) VALUES
(3, 'Супы', 'Холодное', 'Окрошка на кефире', 'Россия', 170);
```

```
select * from hw03.menu final where menu_category='Супы';
```

```
alter table hw03.menu
```

```
update price=price*0.9
```

```
where menu_category='Основное блюдо' and menu_type='Мясные блюда';
```

```
alter table hw03.menu
```

```
delete
```

```
where price>4000;
```

4.Добавьте несколько новых полей в таблицу и удалите два-три существующих.

```
alter table hw03.menu add column if not exists portion_weight UInt8 comment 'Масса порции';
```

```

alter table hw03.menu add column if not exists portion_count UInt8 comment
'Количество порций';
alter table hw03.menu add column if not exists cooked DateTime default now() comment
'Дата и время приготовления';
alter table hw03.menu add column if not exists expired DateTime comment 'Дата и время
истечения срока годности';

```

```

select * from system.columns hw03 where table='menu';

```

```

alter table hw03.menu drop column if exists portion_weight;
alter table hw03.menu drop column if exists portion_count;
alter table hw03.menu drop column if exists expired;

```

5.Выполните выборку данных (select) из любой таблицы из [sample dataset](#)
Взяла в качестве dataset данные о такси

```

CREATE TABLE uk.uk_price_paid_1
(
    `pk` String,
    `price` UInt32,
    `date` Date,
    `postcode1` LowCardinality(String),
    `postcode2` LowCardinality(String),
    `is_new` UInt8,
    `duration` LowCardinality(String),
    `addr1` String,
    `addr2` String,
    `street` LowCardinality(String),
    `locality` LowCardinality(String),
    `town` LowCardinality(String),
    `district` LowCardinality(String),
    `county` LowCardinality(String),
    `a` LowCardinality(String),
    `b` LowCardinality(String)
)
ENGINE = MergeTree
ORDER BY (postcode1,
postcode2,
addr1,
addr2)
SETTINGS index_granularity = 8192;

INSERT INTO uk.uk_price_paid_1
SELECT
    pk,
    price,
    toDate(substring(date, 1, 10)) AS date,
    postcode1,

```

```

        postcode2,
        if(is_new = 'Y', 1, 0) AS is_new,
        duration,
        addr1,
        addr2,
        street,
        locality,
        town,
        district,
        county,
        a, b
FROM file(
    'pp-complete.csv',
    'CSV',
    'pk String,
price UInt32,
date String,
postcode1 String,
postcode2 String,
is_new String,
duration String,
addr1 String,
addr2 String,
street String,
locality String,
town String,
district String,
county String, a String, b String');

```

Соберем сумму в разрезе лет по улице 'London Road':

```

select toStartOfYear(`date`) as start_year sum(price) from uk.uk_price_paid_1 where
street='LONDON ROAD' GROUP by start_year order by start_year;

```

```

clickhouse-server-1 :) select toStartOfYear( date ) as start_year, sum(price) from uk.uk_price_paid_1 where street= 'LONDON ROAD' GROUP by start_year order by start_year;

```

```

SELECT
    toStartOfYear(date) AS start_year,
    sum(price)
FROM uk.uk_price_paid_1
WHERE street = 'LONDON ROAD'
GROUP BY start_year
ORDER BY start_year ASC
Query id: 55d8677a-f63e-4a45-bd87-954abe5d7d0a

```

	start_year	sum(price)
1.	1995-01-01	114523779
2.	1996-01-01	142622658
3.	1997-01-01	197915042
4.	1998-01-01	206096694
5.	1999-01-01	285992435
6.	2000-01-01	274514434
7.	2001-01-01	358339254
8.	2002-01-01	445775322
9.	2003-01-01	454577459
10.	2004-01-01	509664535
11.	2005-01-01	483855869
12.	2006-01-01	669125208
13.	2007-01-01	704756223
14.	2008-01-01	328763397
15.	2009-01-01	313853596
16.	2010-01-01	355818046
17.	2011-01-01	325393691
18.	2012-01-01	337732981
19.	2013-01-01	459108159
20.	2014-01-01	796580749
21.	2015-01-01	1382426760
22.	2016-01-01	2236398802
23.	2017-01-01	2461407228
24.	2018-01-01	2074468242
25.	2019-01-01	2223658250
26.	2020-01-01	2590558258
27.	2021-01-01	2892246310
28.	2022-01-01	3272743314
29.	2023-01-01	1914371868
30.	2024-01-01	1482323192
31.	2025-01-01	82836880

```

31 rows in set. Elapsed: 0.172 sec. Processed 39.60 million rows, 97.42 MB (230.57 million rows/s., 567.17 MB/s.)
Peak memory usage: 2.60 MiB.

```

6.Материализуйте выбранную таблицу, создав её копию в виде отдельной таблицы.

--создаем проекцию для таблицы:

```
ALTER TABLE uk.uk_price_paid 1 ADD projection uk price paid 1 street projection  
(SELECT * ORDER BY street);
```

--заполняем ее данными:

```
ALTER TABLE uk.uk_price_paid_1 MATERIALIZE projection  
uk_price_paid_1_street_projection;
```

--еще раз выполняем тот же самый запрос:

```
select toStartOfYear(`date`) as start_year sum(price) from uk.uk_price_paid_1 where  
street='LONDON ROAD' GROUP by start_year order by start_year;
```

```
SELECT  
    toStartOfYear(date) AS start_year,  
    sum(price)  
FROM uk.uk_price_paid_1  
WHERE street = 'LONDON ROAD'  
GROUP BY start_year  
ORDER BY start_year ASC  
  
Query id: 2ebf96f0-5a18-4963-bfcb-0e86dc096e07  
  
Connecting to localhost:9000 as user click1.  
Connected to ClickHouse server version 25.3.2.
```

	start_year	sum(price)
1.	1995-01-01	114523779
2.	1996-01-01	142622658
3.	1997-01-01	197915042
4.	1998-01-01	206096694
5.	1999-01-01	285992435
6.	2000-01-01	274514434
7.	2001-01-01	358339254
8.	2002-01-01	445775322
9.	2003-01-01	454577459
10.	2004-01-01	509664535
11.	2005-01-01	483855869
12.	2006-01-01	669125208
13.	2007-01-01	704756223
14.	2008-01-01	328763397
15.	2009-01-01	313853596
16.	2010-01-01	355818046
17.	2011-01-01	325393691
18.	2012-01-01	337732981
19.	2013-01-01	459108159
20.	2014-01-01	796580749
21.	2015-01-01	1382426760
22.	2016-01-01	2236398802
23.	2017-01-01	2461407228
24.	2018-01-01	2074468242
25.	2019-01-01	2223658250
26.	2020-01-01	2590558258
27.	2021-01-01	2892246310
28.	2022-01-01	3272743314
29.	2023-01-01	1914371868
30.	2024-01-01	1482323192
31.	2025-01-01	82836880

```
31 rows in set. Elapsed: 0.044 sec. Processed 106.50 thousand rows, 747.98 KB (2.41 million rows/s., 16.91 MB/s.)  
Peak memory usage: 62.23 KiB.
```

Видим, что запрос выполнен в 4 раза быстрее и вместо 39,6 млн записей было проанализировано всего 106,5 тыс записей.

Если выполнить тот же запрос с explain, то видно, что использовалась наша проекция

```

EXPLAIN
SELECT
    toStartOfYear(date) AS start_year,
    sum(price)
FROM uk.uk_price_paid_1
WHERE street = 'LONDON ROAD'
GROUP BY start_year
ORDER BY start_year ASC

```

Query id: 98e1daef-cb29-4fdb-b698-aab2091d4637

```

explain
1. Expression (Project names)
2.   Sorting (Sorting for ORDER BY)
3.     Expression ((Before ORDER BY + Projection))
4.       Aggregating
5.         Filter
6.           ReadFromMergeTree (uk_price_paid_1_street_projection)

```

6 rows in set. Elapsed: 0.005 sec.

8. Попрактикуйтесь с партициями: выполните операции ATTACH, DETACH и DROP. После этого добавьте новые данные в первоначально созданную таблицу.

Создадим новую таблицу для напитков menu_drinks, такую же по структуре, как и основная hw03.menu, заполним ее, потом присоединим ее как партицию к основной таблице hw03.menu и за ненадобностью удалим новую таблицу menu_drinks.

```

CREATE TABLE hw03.menu_drinks
(
    `id` UInt32 COMMENT 'Идентификатор строки',
    `menu_category` LowCardinality(String) COMMENT 'Категория блюда',
    `menu_type` LowCardinality(String) COMMENT 'Тип блюда',
    `name` String COMMENT 'Название блюда',
    `country` LowCardinality(String) DEFAULT 'не задано' COMMENT 'Национальное блюдо',
    `price` UInt32 DEFAULT 0,
    `cooked` DateTime DEFAULT now() COMMENT 'Дата и время приготовления'
)
ENGINE = ReplacingMergeTree
PARTITION BY menu_category
ORDER BY (menu_type,
    country,
    name)
SETTINGS index_granularity = 8192
COMMENT 'Таблица меню напитков';

INSERT INTO hw03.menu_drinks (id, menu_category, menu_type, name, country, price)
VALUES
(7, 'Напитки', 'Чай', 'Молочный уулун', 'Китай', 200),

```

```
(8, 'Напитки', 'Морсы', 'Морс из клюквы', 'Россия', 300);
```

--добавляем промежуточную таблицу как секцию в основную таблицу

```
ALTER TABLE hw03.menu ATTACH PARTITION 'Напитки' FROM hw03.menu_drinks;
```

--удаляем за ненадобностью промежуточную таблицу

--(все данные останутся в основной)

```
drop table hw03.menu_drinks;
```

--сделаем полную выборку из таблицы, чтобы убедиться, что добавилась новая партиция

```
clickhouse-server-1 :) select * from hw03.menu;
```

```
SELECT *  
FROM hw03.menu
```

Query id: 7ed797cc-0ee1-4deb-bf8c-c23d831ffa80

	id	menu_category	menu_type	name	country	price	cooked
1.	8	Напитки	Морсы	Морс из клюквы	Россия	300	2025-04-27 21:23:50
2.	7	Напитки	Чай	Молочный уулун	Китай	200	2025-04-27 21:23:50
3.	2	Десерты	Торт	Шоколадный торт	Франция	300	2025-04-27 21:25:03
4.	4	Супы	Холодное	Окрошка на квасе	Россия	150	2025-04-27 21:25:03
5.	3	Супы	Холодное	Окрошка на кефире	Россия	150	2025-04-27 21:25:03
6.	1	Салаты	Вегетарианское	Салат Цезарь	Италия	500	2025-04-27 21:25:03
7.	3	Супы	Холодное	Окрошка на кефире	Россия	170	2025-04-27 21:25:03
8.	6	Основное блюдо	Мясные блюда	Пельмени	Россия	297	2025-04-27 21:25:03

8 rows in set. Elapsed: 0.008 sec.

Видим, что напитки тоже есть.

Теперь отцепим секцию с напитками:

```
ALTER TABLE hw03.menu DETACH PARTITION 'Напитки';
```

Выполним запрос к БД, увидим, что строки с напитками пропали. В запросе по партициям следующая картина (партиция с напитками не активна):

```
clickhouse-server-1 :) select partition, active from system.parts where table = 'menu';

SELECT
    partition,
    active
FROM system.parts
WHERE `table` = 'menu'

Query id: 5a43a282-0222-45ce-8e51-0bddab7e521d
```

	partition	active
1.	Десерты	1
2.	Основное блюдо	1
3.	Салаты	1
4.	Супы	1
5.	Супы	1
6.	Напитки	0

```
6 rows in set. Elapsed: 0.004 sec.

clickhouse-server-1 :)
```

При выполнении этого же запроса через несколько минут замечаем, что неактивная партиция сама как будто удалась (но физически она есть, пробуем ее вернуть)

```
ALTER TABLE hw03.menu ATTACH PARTITION 'Напитки';
```

```
SELECT
    partition,
    active
FROM system.parts
WHERE `table` = 'menu'

Query id: f0a3697f-86af-4607-9dfa-a355a3688f11
```

	partition	active
1.	Десерты	1
2.	Основное блюдо	1
3.	Салаты	1
4.	Супы	1
5.	Супы	1
6.	Напитки	1

```
6 rows in set. Elapsed: 0.004 sec.
```

Удалим партицию с десертами:

```
ALTER TABLE hw03.menu DROP PARTITION 'Десерты';
```

Проверим наличие данных в таблице:

```
select * from hw03.menu; --строк с десертами нет
```

Проверим, что в таблице system.parts:

```
select partition, active, rows, remove_time FROM system.parts where table='menu';
```

```
SELECT
    partition,
    active,
    rows,
    remove_time
FROM system.parts
WHERE `table` = 'menu'
```

Query id: d5249259-7e0c-4bf6-93d0-76b99c1b4652

	partition	active	rows	remove_time
1.	Десерты	0	0	2025-04-27 21:47:12
2.	Основное блюдо	1	1	1970-01-01 00:00:00
3.	Салаты	1	1	1970-01-01 00:00:00
4.	Супы	1	2	1970-01-01 00:00:00
5.	Супы	1	1	1970-01-01 00:00:00
6.	Напитки	1	2	1970-01-01 00:00:00

Видим, что партиция с десертами не активна, в ней 0 строк и заполнена дата удаления.

Попробуем прикрепить удаленную партицию:

```
ALTER TABLE hw03.menu ATTACH PARTITION 'Десерты';
```

Видим, что в system.parts появилась партиция, но строк в ней нет.

Если сделать `select * from hw03.menu`, то данных по напиткам не появилось, чего мы и ожидали


```
clickhouse-server-1 :) select partition, active, rows, remove_time FROM system.parts where table='menu';
```

```
SELECT
    partition,
    active,
    rows,
    remove_time
FROM system.parts
WHERE `table` = 'menu'
```

Query id: 86d31b13-1773-486a-8153-5205c9ac7536

	partition	active	rows	remove_time
1.	Десерты	0	0	2025-04-27 21:47:12
2.	Основное блюдо	1	1	1970-01-01 00:00:00
3.	Салаты	1	1	1970-01-01 00:00:00
4.	Супы	1	2	1970-01-01 00:00:00
5.	Супы	1	1	1970-01-01 00:00:00
6.	Напитки	1	2	1970-01-01 00:00:00

6 rows in set. Elapsed: 0.003 sec.

```
clickhouse-server-1 :) select * from hw03.menu;
```

```
SELECT *
FROM hw03.menu
```

Query id: 3d4ba319-62a2-48bf-a70b-2cdfaa38ea3e

	id	menu_category	menu_type	name	country	price	cooked
1.	8	Напитки	Морсы	Морс из клюквы	Россия	300	2025-04-27 21:23:50
2.	7	Напитки	Чай	Молочный уулун	Китай	200	2025-04-27 21:23:50
3.	6	Основное блюдо	Мясные блюда	Пельмени	Россия	297	2025-04-27 21:53:57
4.	4	Супы	Холодное	Окрошка на квасе	Россия	150	2025-04-27 21:53:57
5.	3	Супы	Холодное	Окрошка на кефире	Россия	150	2025-04-27 21:53:57
6.	3	Супы	Холодное	Окрошка на кефире	Россия	170	2025-04-27 21:53:57
7.	1	Салаты	Вегетарианское	Салат Цезарь	Италия	500	2025-04-27 21:53:57

7 rows in set. Elapsed: 0.007 sec.

```

SELECT
    partition,
    active,
    rows,
    remove_time
FROM system.parts
WHERE `table` = 'menu'

```

Query id: 86d31b13-1773-486a-8153-5205c9ac7536

	partition	active	rows	remove_time
1.	Десерты	0	0	2025-04-27 21:47:12
2.	Основное блюдо	1	1	1970-01-01 00:00:00
3.	Салаты	1	1	1970-01-01 00:00:00
4.	Супы	1	2	1970-01-01 00:00:00
5.	Супы	1	1	1970-01-01 00:00:00
6.	Напитки	1	2	1970-01-01 00:00:00

6 rows in set. Elapsed: 0.003 sec.

clickhouse-server-1 :) select * from hw03.menu;

```

SELECT *
FROM hw03.menu

```

Query id: 3d4ba319-62a2-48bf-a70b-2cdfaa38ea3e

	id	menu_category	menu_type	name	country	price	cooked
1.	8	Напитки	Морсы	Морс из клюквы	Россия	300	2025-04-27 21:23:50
2.	7	Напитки	Чай	Молочный уулун	Китай	200	2025-04-27 21:23:50
3.	6	Основное блюдо	Мясные блюда	Пельмени	Россия	297	2025-04-27 21:53:57
4.	4	Супы	Холодное	Окрошка на квасе	Россия	150	2025-04-27 21:53:57
5.	3	Супы	Холодное	Окрошка на кефире	Россия	150	2025-04-27 21:53:57
6.	3	Супы	Холодное	Окрошка на кефире	Россия	170	2025-04-27 21:53:57
7.	1	Салаты	Вегетарианское	Салат Цезарь	Италия	500	2025-04-27 21:53:57

7 rows in set. Elapsed: 0.007 sec.